

Syllabus: Surrogate Models for Process Decision-Making

An eight-week graduate-entry course connecting process prediction, physics-informed learning, and optimization embedding.

Course design and notes: Sunwoo Kim · Graduate entry · Updated 2026-10-06

Course purpose

A process surrogate approximates the response of a process model or measurements. A decision system also needs a choice of operating variables, an objective, constraints, and a way to check the selected operating point. This course connects these tasks without treating prediction accuracy as evidence of good decisions.

Designed by Sunwoo Kim as an independent, self-paced learning course for chemical engineers entering graduate study. The materials combine original explanations and exercises with cited research papers.

Audience and prerequisites

Basic Python, derivatives, matrix operations, linear algebra, and material balances are expected. Prior neural-network or mathematical-programming research experience is not required. LP/MILP, convexity, and KKT conditions are introduced when they are needed.

Suggested weekly workload: 90 minutes of theory, 60–90 minutes of guided coding, and 2–3 hours of reading or exercises. These are study recommendations, not scheduled classes.

Learning outcomes

By the end of the course, learners should be able to:

1. Define decision variables, fixed context, states, outputs, units, and a valid operating domain.
2. Build and evaluate steady-state vector-to-vector and dynamic sequence-to-sequence surrogates, including conditioning on future control inputs.
3. Compare data-driven, soft physics-informed, and hard linear-equality-constrained models using both prediction errors and constraint residuals.
4. Derive a valid MILP graph formulation for a frozen ReLU network and check its numerical agreement with a forward pass.
5. Explain when a ReLU ICNN/PICNN admits an LP epigraph formulation and when that formulation is not equivalent to the intended problem.
6. Report simulator revalidation, feasibility, decision quality, bounds, solver termination, and computation time separately.

Eight-week sequence

Week	Topic and scope	Required output
1	From process prediction to decisions. Variables, units, operating domains, data splits, optimization, and simulator revalidation.	A CSTR problem specification and a reproducible prediction-to-decision baseline.
2	Steady-state vector-to-vector surrogates. Multi-output MLPs, scaling, sampling, output errors, and balance residuals.	A trained multi-output MLP with a documented domain and held-out evaluation.
3	Dynamic sequence-to-sequence surrogates. Window MLPs, state transitions, RNN/LSTM, direct prediction, rollout, and trajectory splits.	One-step and multi-step predictions conditioned on a candidate future input sequence.
4	Physics-informed learning and KKT-hPINN. Residual losses, automatic differentiation, linear equality projection, KKT derivation, and rank conditions.	An MLP/PINN/KKT-hPINN comparison that separates accuracy from constraint satisfaction.
5	Exact MILP embedding of ReLU networks. Network graphs, binary variables, Big-M derivation, valid activation bounds, and scaling.	A hand-built embedding whose outputs match the frozen network at fixed inputs.
6	Embedding in practice: bounds, OMLT, and horizons. Bound tightening, LP relaxations, MIP gaps, OMLT, affine projection layers, and finite-horizon ReLU models.	A formulation and solve-time comparison with forward-pass revalidation.
7	ICNN/PICNN and conditional LP reformulation. Convexity conditions, epigraphs, fixed context, constraint direction, and equality counterexamples.	An LP on a verified convex example and an explicit list of equivalence conditions.
8	A complete process decision workflow. Decision-point errors, extrapolation, feasibility, economic performance, simulator rechecks, and targeted data collection.	A reproducible report covering model error, decision quality, violations, and computation time.

Weeks 2–4 focus on building and evaluating models. Weeks 5–7 focus on optimization embedding. Week 8 joins the two parts. The syllabus and **Week 1** are available now; Weeks 2–8 describe planned content and do not yet have released lecture notes.

Reading assignments

Week	Reading guidance
1	Boyd & Vandenberghe, Chapters 2–4 (selected sections); OMLT introduction.
2	KKT-hPINN, Sections 1–3 (MLP and process-surrogate setup).
3	Sutskever et al., Sections 1–2; DeepONet introduction (optional).
4	Raissi et al., PINN formulation; Chen et al., Section 3 and selected case studies.

Week	Reading guidance
5	Anderson et al., formulation preliminaries and ReLU formulations.
6	OMLT paper and documentation; Anderson et al., stronger formulations (selected).
7	Amos et al., Section 3 and Supplement B.
8	Revisit the assumptions and validation results from Weeks 1–7.

The reading list is for selected sections, not six full-paper assignments each week. Advanced formulation proofs and operator-learning extensions are optional.

Core sources

- **Convex optimization background:** S. Boyd and L. Vandenberghe, *Convex Optimization* (2004). Read selected parts of Chapters 2–4 for convex sets, convex functions, epigraphs, and optimization problems; return to Chapter 5 for duality and KKT. **Official book and downloads** · **EE364a slides**.
- **ICNN/PICNN:** B. Amos, L. Xu, and J. Z. Kolter, *Input Convex Neural Networks*, ICML (2017). Section 3 gives architecture conditions; Supplement B gives LP inference for ReLU/linear units. **Paper** · **PDF** · **Supplement**.
- **ReLU formulations:** R. Anderson, J. Huchette, W. Ma, C. Tjandraatmadja, and J. P. Vielma, *Strong mixed-integer programming formulations for trained neural networks*, Mathematical Programming (2020). The public manuscript was first submitted in 2018 and revised in 2020. **Public manuscript**.
- **Embedding software:** F. Ceccon, J. Jalving, J. Haddad, A. Thebelt, C. Tsay, C. D. Laird, and R. Misener, *OMLT: Optimization & Machine Learning Toolkit*, JMLR 23(349), 1–8 (2022). **Paper and PDF** · **Code** · **Documentation**.
- **PINN:** M. Raissi, P. Perdikaris, and G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics 378, 686–707 (2019). **Published paper**. The related 2017 Part I manuscript is a freely available introduction; it is not the identical final journal version. **2017 Part I**.
- **KKT-hPINN:** H. Chen, G. E. Constante Flores, and C. Li, *Physics-informed neural networks with hard linear equality constraints*, Computers & Chemical Engineering 189, 108764 (2024). **Published paper** · **Public manuscript** · **Code and datasets**.
- **Sequence-learning background:** I. Sutskever, O. Vinyals, and Q. V. Le, *Sequence to Sequence Learning with Neural Networks*, NeurIPS (2014). Its application is translation; our process-trajectory exercises are separate educational examples. **Paper**.

Optional extensions: **DeepONet, Nature Machine Intelligence (2021)**, for input-function to output-function mappings; **KKT-Hardnet public manuscript (2025)**, published in **C&CE (2026)**, for nonlinear equality and inequality projection.

Process examples and software

The main educational example is a small isothermal CSTR with a first-order reaction. Week 1 supplies its equations, synthetic parameters, steady-state map, and dynamic simulation. It can be run without Aspen or a plant dataset. The public KKT-hPINN datasets provide a separate research-reproduction exercise; they are not generated by our teaching simulator.

Convexity and LP equivalence are first checked on a separate example with a known convex target. Convexity of an ICNN approximation does not establish convexity of an actual CSTR or of every output of a flowsheet.

Week 1 requires Python, NumPy, and Matplotlib. Later materials will introduce PyTorch and Pyomo/OMLT with a suitable solver. A notebook and a standalone Python script are both supplied so that Jupyter is optional for the first lab.

What “exact” and “hard” mean here

- **Exact embedding:** a formulation represents the frozen learned function over the stated domain. It does not remove approximation error relative to the physical process. MILP status also depends on the rest of the objective and constraints.
- **LP reformulation:** ReLU/linear ICNN structure, fixed PICNN context, valid nonnegative propagation, and appropriate epigraph usage matter. Arbitrary output equalities, reversed inequalities, smooth activations, or additional nonlinear constraints can invalidate an LP claim.
- **Hard constraints:** KKT-hPINN enforces the specified consistent linear equalities under its matrix assumptions, up to numerical arithmetic. It does not automatically enforce positivity, nonlinear thermodynamics, stability, or every operating limit.
- **Optimization with physics-informed models:** training losses do not determine the algebraic class of the frozen predictor. A ReLU backbone followed by a fixed affine equality projection remains piecewise affine; a smooth or nonlinear projection can change the required formulation.

Exercises and final project

Each week pairs a derivation or problem specification with an executable check. The final report should state the data domain, split method, model architecture, frozen weights, scaling, formulation assumptions, solver and termination status, and simulator used for revalidation.

Evaluate prediction error and constraint violations at held-out points and at selected decisions. Compare economic performance using the reference process model. If a grid or local solver is the benchmark, call it a grid or local benchmark; do not call it a certified global process optimum.

For self-study, assess the final project on four dimensions: correctness of the mathematical formulation, reproducibility, evidence about decision quality, and honesty about unresolved error or feasibility. An incomplete solve or a failed operating point is useful evidence when it is reported accurately.